运算符和常用表达式

1

什么是运算符和表达式

- 运算符：实现运算的符号
- 表达式：能被求值的JavaScript语句，通常由操作数和运算符构成，一些JS内置函数也是表达式

5 % 3

操作数 运算符 操作数

表达式

```
Math.ceil(3.6)
```

2

运算符的种类

- 算术运算符　比如：3 + 3、Math.ceil(3.5)
- 关系运算符　比如：3 > 2
- 逻辑运算符　比如：false && true
- 赋值运算符　比如：a += 3

3

算术运算符

4

算术运算符

意义	运算符
加	+
减	-
乘	*
除	/
取余	%

5

加减乘除的符号

- 加减的符号和数学一致，乘法是*号，除法是/号

6

运算顺序

- 乘除法的优先级要高于加减法
- 乘法和除法是平级的，加法和减法是平级的
- 必要时可以使用圆括号来改变运算的顺序

```
1 + 2 * 3                // 7
(1 + 2) * 3              // 9
1 + 2 / 4                // 1.5
((1 + 2) * 3 + 4) / 2    // 6.5
```

7

加号的两种作用

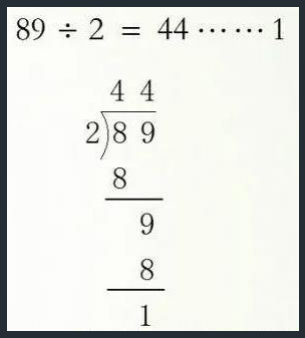
- 加号有“加法”和“连字符”两种作用
- 如果加号两边的操作数有字符串，则加号为连字符

```
1 + 2        // 3
1 + '2'      // '12'
'1' + '2'    // '12'
```

8

取余运算

- 取余运算也叫作“求模运算”，用百分号%表示
- a % b表示求a除以b的余数，它不关心整数部分，只关心余数




```
89 % 2        // 1
```

9

取余运算

```
11 % 4;        //3，因为11除以4余数是3
20 % 6;        //2，因为20除以6余数是2
12 % 3;        //0，因为能够整除，余数是0
9 % 3;         //0，因为能够整除，余数是0
3 % 9;         //3，因为商0，余数是3
```

10

IEEE754

- 在JavaScript中，有些小数的数学运算不是很精准，比如0.1+0.2，结果是0.30000000000000004
- JavaScript使用了IEEE754二进制浮点数算术标准，这会使一些个别的小数运算产生“丢失精度”问题
- 解决方法我们后面课程介绍

11

小案例：计算复杂算式

12

计算复杂算式

- 能够将复杂算式变为JS表达式，是编程基本功

$$\frac{2+5^3}{3} \times \sqrt{\frac{3050-8}{2}}$$

13

幂和开根号

- JavaScript中没有提供幂计算、开根号的运算符。需要使用Math对象的相关方法进行计算

```
Math.pow(2, 3)      // 8     2^3
Math.pow(3, 2)      // 9     3^2

Math.sqrt(81)       // 9     √81
Math.sqrt(-81)      // NaN
```

14

小案例：计算正六边形面积

15

计算正六边形面积

- 题目：由用户输入正六边形的边长，请计算并显示正六边形的面积

$$s = \frac{3\sqrt{3}}{2}a^2$$

16

如何实现用户输入

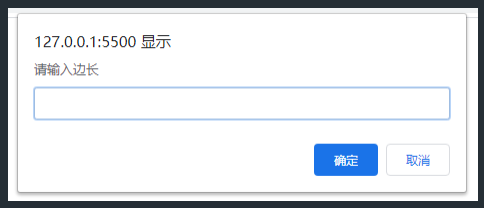
- JavaScript内置了prompt()函数，可以提示用户输入指定的数据

17

如何实现用户输入

- JavaScript内置了prompt()函数，可以提示用户输入指定的数据

```
var a = prompt('请输入边长');
```



18

输入、计算、输出

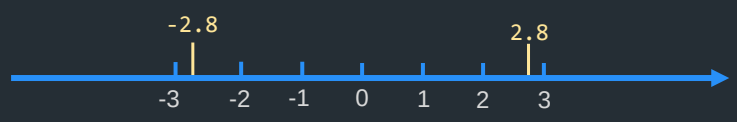
19

向上取整和向下取整

20

向上取整和向下取整

- 向上取整：在数轴的正方向上，寻找第一个整数
  比如2.8向上取整结果是3，-2.8向上取整结果是-2
- 向下取整：在数轴的反方向上，寻找第一个整数
  - 比如2.8向下取整结果是2，-2.8向下取整结果是-3



21

Math.ceil()和Math.floor()

- Math.ceil() 向上取整
- Math.floor() 向下取整

22

parseInt()和parseFloat()函数

23

parseInt()函数

- JavaScript内置的parseInt()函数，功能是将数字或者字符串变为整数

24

parseInt()函数

- parseInt()的工作机理很简单粗暴：直接截断小数点，暴力取整

```
parseInt(3.4)     // 3
parseInt(3.9)     // 3
parseInt(3)       // 3
parseInt(-3.4)    // -3
parseInt(-3.9)    // -3
parseInt(-3)      // -3
```

- 即，parseInt()在正数部分是向下取整，在负数部分是向上取整

25

净化尾文字

- parseInt()函数可以将字符串变为数字（且是整数），并且有“净化尾文字”功能

```
parseInt('3.9')         // 3
parseInt('3.4')         // 3
parseInt('-3.9')        // -3
parseInt('-3.4')        // -3

parseInt('365天')       // 365
parseInt('365个人')     // 365
parseInt('365.66天')    // 365
parseInt('3e5')         // 3
```

26

parseInt()结果可能是NaN

- 下面这些情况，使用parseInt()结果是NaN

```
parseInt('a123')        // NaN
parseInt('abc')         // NaN
parseInt('')            // NaN
parseInt(undefined)     // NaN
parseInt(null)          // NaN
parseInt(Infinity)      // NaN
parseInt(NaN)           // NaN
```

27

parseInt()可以添加进制参数

- parseInt()可以接受第二个参数，用来设置进制

```
parseInt('1111', 2)     // 15
parseInt('f', 16)       // 15
parseInt('17', 8)       // 15
```

28

parseFloat()函数

- JavaScript内置的parseFloat()函数，功能是将字符串变为小数，同样的，它也有尾部文字净化功能

29

parseFloat()函数

```
parseFloat('3.2')        // 3.2
parseFloat('3.9')        // 3.9
parseFloat('3')          // 3
parseFloat('3.8年')      // 3.8年
parseFloat('3.8.9')      // 3.8
parseFloat('a3')         // NaN
parseFloat('')           // NaN
parseFloat(undefined)    // NaN
parseFloat(null)         // NaN
parseFloat(Infinity)     // Infinity
parseFloat(NaN)          // NaN
```

30

使用Number()函数实现数据类型转换

31

Number()函数

```
Number('123');          // 123
Number('123.4');        // 123.4
Number('123年');        // NaN
Number('2e3');          // 2000
Number('');             // 0
Number('NaN');          // NaN
```

字符串 → 数字
纯数字字符串能变为数字，不是纯数字的字符串将转为NaN

```
Number(true);           // 1
Number(false);          // 0
```

布尔值 → 数字
true变为1，false变为0

```
Number(undefined);      // NaN
Number(null);           // 0
```

undefined和null → 数字
undefined变为NaN，null变为0

32

小案例：小小加法计算器

33

小案例：小小加法计算器

- 题目：依次弹出两个prompt框，提示用户依次输入两个数字，然后弹出这两个数字的和
- 这个题目涉及到的一个小知识点：prompt接收到的值，是string，而不是number，如果要进行加法计算，必须先转为数字类型值

34

数字的隐式转换

35

数字的隐式转换

- 隐式类型转换：如果参与数学运算的某操作数不是数字型，那么JavaScript会自动将此操作数转换为数字型

```
'13' - '4';              // 9
'13' * '4';              // 52
'13' / '4';              // 3.25
'13' % '4';              // 1
Math.pow('3', '2');      // 9
```

36

数字的隐式转换

- 隐式类型转换的本质是内部调用Number()函数

字符串 → 数字
纯数字字符串能转为数字，不是纯数字的字符串将转为NaN

```
true * 6          // 6
false * 6         // 0
3 * '2天'         // NaN
3 * '2e3'         // 6000
undefined * 4     // NaN
null * 4          // 0
```

布尔值 → 数字
true转为1，false转为0

undefined和null → 数字
undefined转为NaN，null转为0

37

关于加号的隐式转换

- 加号两边如果有字符串，那么此时就会被当做连字符
- 否则，（即加号两边没有字符串），那么此时就会被当做加号

```
true + true        // 2
5 + true           // 6
'5' + true         // 5true
'5' + undefined    // 5undefined
```

加号两边没有字符串，加号

加号两边没有字符串，加号

加号两边有字符串，连字符

加号两边有字符串，连字符

38

'5' + true

- 连字操作时，操作符会被自动调用String()函数转为字符串类型
- String()函数的转换规则特别傻白甜，就是它长成什么样，就转成什么样的字符串

```
String(123)          // '123'
String(2e3)          // '2000'
String(NaN)          // 'NaN'
String(Infinity)     // 'Infinity'
String(true)         // 'true'
String(false)        // 'false'
String(undefined)    // 'undefined'
String(null)         // 'null'
```

```
'5' + true       // 5true
```

加号两边有字符串，所以加号是连字符。此时系统将用String()函数转换它的操作数。true会被转为字符串'true'，所以最终结果是'5true'

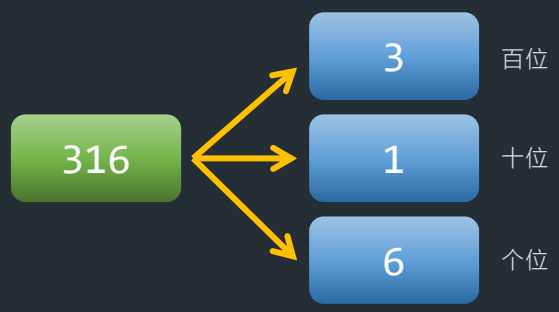
39

小案例：计算3位数各数位之和

40

小案例：计算3位数各数位之和

题目：用户输入一个三位数，程序计算三位数各个数位的总和，并弹出结果。比如用户输入316，则弹出10，因为3+1+6等于10。

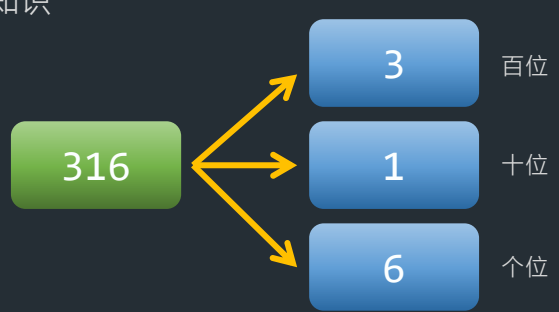


41

如何拆出百位、十位、个位?

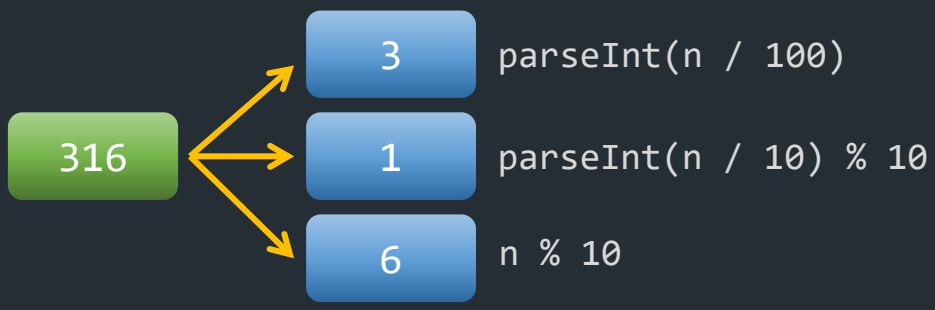
思路1：数学方法，用巧妙的数学方法得到用户输入数字的百位、十位和个位，这里涉及parseInt()、取余等知识

思路2：字符串方法，用字符串的charAt()方法或者[]直接得到某位的值，这里涉及类型转换的知识



42

思路1：数学方法



43

关系运算符

44

关系运算符

意义	运算符
大于	>
小于	<
大于或等于	>=
小于或等于	<=
等于	==
不等于	!=
全等于	===
不全等于	!==

45

大于和小于

- 大于>、小于<两个符号和数学相同
- “大于等于”运算符是>=，“小于等于”运算符是<=

```
8 > 5      // true
7 < 4      // false
8 >= 7     // true
8 >= 8     // true
8 <= 8     // true
8 <= 11    // true
```

46

判断是否相等

- 两个等号==用来判断值是否相等，一个等号=是赋值，萌新选手很容易错

```
3 == 3                // true
'考拉' == '考拉'      // true

3 = 3                 // 报错
'考拉' = '考拉'       // 报错
```

47

相等和全等

- ==不比较类型，===比较类型

```
5 == '5';             // true
5 === '5';            // false

1 == true             // true
1 === true            // false

0 == false            // true
0 === false           // false
```

48

混乱的特殊值比较

- NaN不自等，这个知识点很关键，我们后面会单独用课时介绍
- undefined和0不等，null和0也不等
- 诡异的是undefined和null相同

49

不相等和不全等

- !=表示不相等，!==表示不全等

```
5 != 6          // true
5 !== 6         // true
5 != '5'        // false
5 !== '5'       // true
```

50

不要连比

- 判断变量a是不是介于3到15之间，应该怎么写呢？

```
3 <= a <= 15
```

```
3 <= a && a <= 15
```

51

逻辑运算符

52

逻辑运算符

意义	运算符
非	!
与	&&
或	||

53

非运算

- !表示“非”，也可以称为“置反运算”
- !是一个“单目运算符”，只需要一个操作数
- 置反运算的结果一定是布尔值

54

非运算

```
!true        // false
!false       // true
!0           // true
!undefined   // true
!''          // true
!'kaola'     // false

!!true       // true
!!0          // false
!!''         // false
!!'kaola'    // true
```

55

与运算

- &&表示“且”，称为“与运算”，口诀：都真才真

1+1等于2 且 月亮是三角形的

真 且 假

最终结果：假

56

与运算 - “都真才真”

```
true && true          // true
true && false         // false
false && true         // false
false && false        // false
```

57

或运算

- ||表示“或者”，称为“或运算”，口诀：有真就真

1+1等于2 或 月亮是三角形的

真 或 假

最终结果：真

58

Esc
F1
F2
F3
F4
F5
F6
F7
F8
F9
F10
F11
F12
Print Screen
Scroll Lock
Pause Break
Tab
Caps Lock
Enter
Shift
Shift
Ctrl
Alt
Alt
Ctrl
Insert
Home
Page Up
Delete
End
Page Down
Num Lock
Enter
Del

59

60

或运算 - “有真就真”

```
true || true        //true
true || false       //true
false || true       //true
false || false      //false
```

61

短路计算

62

短路计算

- 请思考下面表达式的运算结果

```
3 && 6   // 6
```

63

短路计算

```
a && b
```

真

b真总结果就真
b假总结果就假

b就是总结果

```
3 && 0          // 0
3 && 9          // 9
3 && 'abc'      // 'abc'
3 && ''         // ''
3 && undefined  // undefined
```

64

短路计算

a && b

假

不用看b了
被“短路”

a就是总结果

```
0 && 9              // 0
'' && 8             // ''
'' && 0             // ''
false && 6          // false
false && 0          // false
undefined && true   // undefined
undefined && false  // undefined
```

65

短路计算

a || b

真

不用看b了
b被短路

a就是总结果

```
3 || 0          // 3
3 || 9          // 3
3 || 'abc'      // 3
3 || ''         // 3
3 || undefined  // 3
```

66

短路计算

a || b

假

b真总结果就真
b假总结果就假

b就是总结果

```
0 || 9                // 9
'' ||  8              // 8
'' ||  0              // 0
false || 6            // 6
false || 0            // 0
undefined || true     // true
undefined || false    // false
```

67

实际上，且、或的口诀也符合短路计算

```
true && true            // true
true && false           // false
false && true           // false
false && false          // false

true || true        //true
true || false       //true
false || true       //true
false || false      //false
```

68

短路计算题目

```
3 && 6
15 && undefined
0 || 6
null && 2
'' && 16
null || undefined
NaN && undefined
'a' || 'b'
NaN || null
undefined && 15
3 || 6
```

69

NaN不自等和isNaN()函数

70

NaN不自等

- NaN不自等，人家就这么规定的，没辙

```
NaN == NaN      // false
NaN === NaN     // false
```

- 因此，判断一个变量的值是否是NaN挺难的

```
var a = Number(prompt('请输入一个数字'));
if (a != NaN) {

}
```

71

isNaN()函数

- isNaN()函数可以用来判断变量值是否为NaN

```
isNaN(NaN)      // true
isNaN(5)        // false
```

- 但isNaN()也是大坑！isNaN()函数的机理：

  只要某值用Number()转为数字的结果是NaN，则isNaN()函数就会得到true

```
isNaN(undefined)    // true
isNaN('3天')        // true
isNaN(false)        // false
isNaN(null)         // false
```

72

判断NaN

- 解决NaN的判断问题其实也很简单，就是连带类型进行比较

```
var a = Number(prompt('请输入一个数字'));
if (typeof a == 'number' && !isNaN(a)) {

}
```

73

逻辑运算的运算顺序

74

逻辑运算的运算顺序

- 逻辑运算的优先级是：非 → 与 → 或

```
!true || true        // true
3 && 4 || 5 && 6     // 4
```

75

综合运算顺序

76

综合运算顺序

- 运算顺序： 非运算 → 数学运算 → 关系运算 → 逻辑运算

```
3 > 2 && !8 > 3 + 4  // false
```

77

袋鼠厂2019年校园秋招题目

```
3 + undefined || '' >= false - null && 9
```

78

小案例：闰年判断

79

闰年判断方法

- 闰年条件（符合以下条件之一即可）

  1. 能被4整除且不能被100整除
  2. 能被100整除也能被400整除

1892年 是　1896年 是　1900年 非　1904年 是

1992年 是　1996年 是　2000年 是　2004年 是

80

赋值运算符

81

赋值运算符

意义	运算符
赋值	=
快捷赋值	+=
	-=
	*=
	/=
	%=
自增运算	++
自减运算	--

82

赋值运算符

- JS中，=表示赋值，==判断是否相等（不判断类型），===判断是否全等
- 赋值运算符会将等号右边的数值，赋给等号左边的变量

```
var a = 5;
```

83

快捷赋值

- 快捷赋值运算符表示在原数值基础上进一步进行运算

```
var a = 3;
a += 5;              // 等价于a = a + 5
console.log(a);      // 8

var b = 14;
b /= 2;              // 等价于b = b / 2
console.log(b);      // 7
```

84

快捷赋值

- 快捷赋值运算符表示在原数值基础上进一步进行运算

```
var c = 16;
c %= 3;              // 等价于c = c % 3
console.log(c);      // 1
```

85

自增/自减运算符

- ++和--是自增/自减运算符，表示在自身基础上加1或减1

```
var a = 3;
a++;
console.log(a);       // 4
```

86

a++和++a的区别

a++ 先用再加

```
var a = 3;
var b = a++;
console.log(b);        // 3
console.log(a);        // 4
```

++a 先加再用

```
var a = 3;
var b = ++a;
console.log(b);        // 4
console.log(a);        // 4
```

87

赋值运算也产生值

88

赋值运算也产生值

- 赋值运算也产生值，等号后面的值将作为“赋值运算的值”

```
var a;
console.log(a = 4);     // 4
```

- 这就意味着，可以连续使用赋值运算符

```
var a, b, c;
a = b = c = 15;
console.log(a);         // 15
console.log(b);         // 15
console.log(c);         // 15
```